

UNIVERSITY PARTNER



Sentiment Analysis of Movie Review using RNN, LSTM, and Word2Vec Embeddings

6CS012 Artificial Intelligence and Machine Learning

University ID	: 2331250
Student Name	: Riddhi Maskey
Group	: L6CG13
Tutor	: Durga Pokharel
Module Leader	: Simon Giri
Submitted on	: <05/13/2025>
Total no. of words	: 1773

Abstract

This research paper main focuses on the analysis of the review of the movie that has launched in order to determine whether it was positive or negative. The study paper has utilized the pretrained Word 2Vec embedding's to make the representation of the word and the classification performance upgraded for deeper understandings with two models such as Recurrent Neural Networks(RNN) and Long Short-Term Memory(LSTM) networks. According to the results, Long Short-Term Memory(LSTM) outperformed than others models with the greater accuracy rate of about 88.8%. The benefit of using Long Short-Term Memory(LSTM) models has been successful in implementing pretrained embedding's and also in the sequential processing text input. In the future, if needed for the better performance of the model the attention mechanisms can have used as a complex theory.

Table of Contents

1	Introduction	1
2	Data set.....	1
2.1	Source:	1
2.2	Data size	1
2.3	Classes	2
2.4	Preprocessing Steps.....	2
3	Methodology	2
3.1	Text Preprocessing	2
3.2	Model Architectures	3
i.	RNN	3
3.3	Loss Function.....	6
3.4	Optimizer.....	6
3.5	Hyperparameters	6
4	Experiments and Results	6
4.1	RNN vs LSTM Performance.....	6
4.2	Computational Efficiency	7
4.3	Training with Different Embeddings	7
4.4	Model Evaluation	7
5	Conclusion and Future Work.....	10

Table of Figures

Figure 1: Total number of reviews per data sets	2
Figure 2: Distribution of sequence length on training data	3
Figure 3: Accuracy and loss of RNN	4
Figure 4: Accuracy and Loss of LSTM	5
Figure 5: Accuracy and loss of LSTM and Word2Vec	5
Figure 6: Test accuracy and loss of three architecture.....	8
Figure 7: Classification report of three models	8
Figure 8: Confusion matrix of RNN	9
Figure 9: Confusion matrix of LSTM.....	9
Figure 10: Confusion matrix of LSTM and Word2Vec	10

1 Introduction

Sentiment Analysis is a Natural Language Processing (NLP) Technique which is used to identify the emotional tone of text by helping to interpret opinions, attitudes and emotions expressed in the written languages. The main objective of sentiment analysis is to determine whether the sentiment is positive or Negative. The classifications of the text can be dangerous for many businesses to observe the social media posts, reviews and feedback of the customers. Both the deep learning models as Recurrent Neural Networks(RNN) and Long Short-Term Memory(LSTM) are utilized for the data sequence as they provide the contextual information. But, by extra number of memory cells, Long Short-Term Memory(LSTM)are causing the problem of the gradients vanishing which plague conventional RNNs. Alongside the other two models Word2Vec embeddings also can facilitate some tasks of the text turns out to the dense vectors which can maintain some semantic links with the representations of the input in the deeper understandings.

2 Data set

2.1 Source:

The dataset that is used for this project is the Movie Review dataset, which was selected from the recommended dataset for the text classification tasks, which contains features like review and sentiment.

2.2 Data size

In this research paper, the various sizes of data have been collected as Training Data, Validation Data, and Testing Data, along with the exact number of the reviews got as the responsive while doing the movie reviews as 35000 reviews on Training Data, 5000 reviews on Validation Data and 10000 reviews on Testing Data.

```
Total reviews on training data: 35000
Total reviews on testing data: 10000
Total reviews on validation data: 5000
```

Figure 1: Total number of reviews per data sets

2.3 Classes

Both the positive and negative alongside binary classification.

2.4 Preprocessing Steps

The preprocessing steps are mentioned below:

- Clearing Text by removing punctuation and special characters.
- Making all characters lowercase.
- Segmenting sentences into words
- Standardizing sequence lengths by padding

3 Methodology

3.1 Text Preprocessing

❖ Clearing

- URLs excluded and emojis and @user added
- Converting text to lowercase
- Making all input sequences the same length by adding padding to the shorter ones.
- Performed lemmatization to reduce words to their base form (e.g., 'running' to 'run')

❖ Segmentation

- Transformed sentences into numerical word sequences with Keras Tokenizer
- Assigned each word a unique integer identifier

❖ Standardizing

- Uniform input length

- Maximum length size on data set analysis

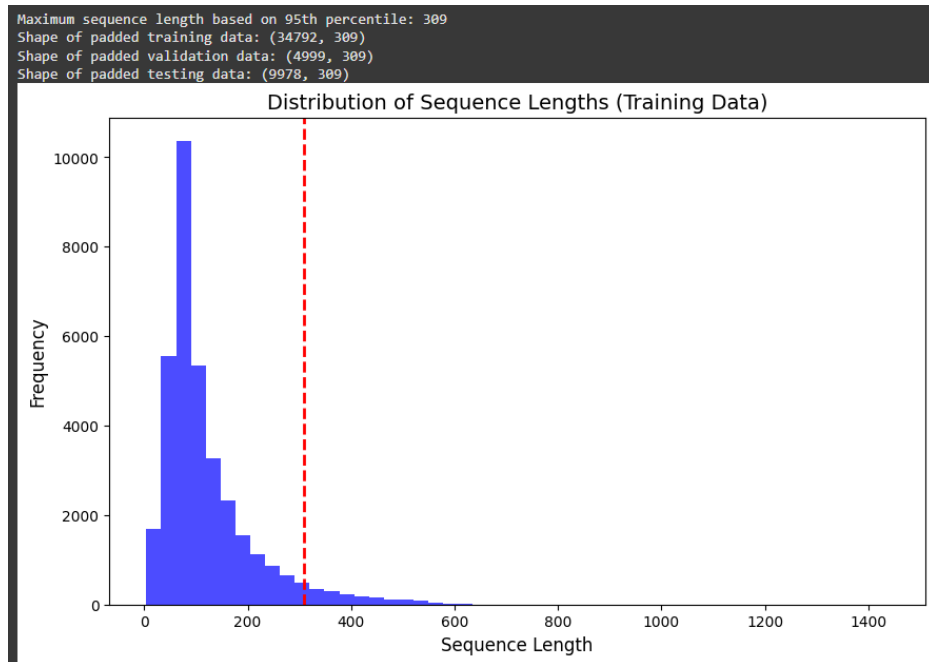


Figure 2: Distribution of sequence length on training data

3.2 Model Architectures

i. RNN

Recurrent Neural Networks (RNNs) process sequential data and consist of several layers:

- ❖ Embedded Layer: Converts categorical data (like words) into numerical vectors, capturing relationships between them.
- ❖ Simpler Layer: Handles basic processing tasks within the network before moving to more complex operations.
- ❖ Dense Layer: Fully connects previous neurons to this layer, aggregating features learned and preparing for predictions.
- ❖ Output Layer: Produces the final prediction or classification based on the inputs processed by the earlier layers.

These layers work together to enable RNNs to handle tasks involving sequences effectively.

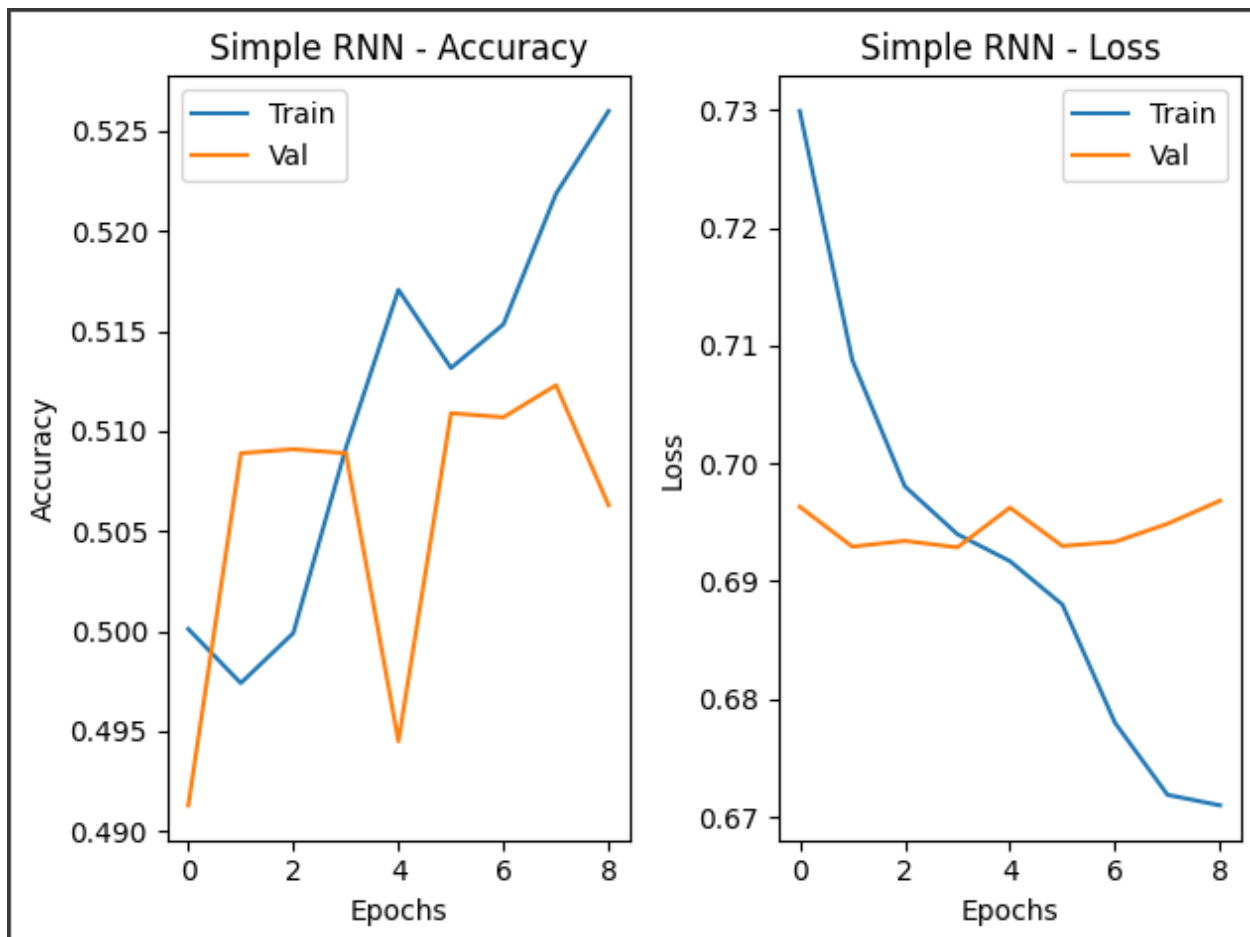


Figure 3: Accuracy and loss of RNN

ii. LSTM

To deal with issues like vanishing gradients that occur in regular RNNs, the second model used Long Short-Term Memory (LSTM) layers instead of simple RNNs. It also included a layer that learns word representations. LSTM layers are better suited for handling long-term relationships in sequences of data.

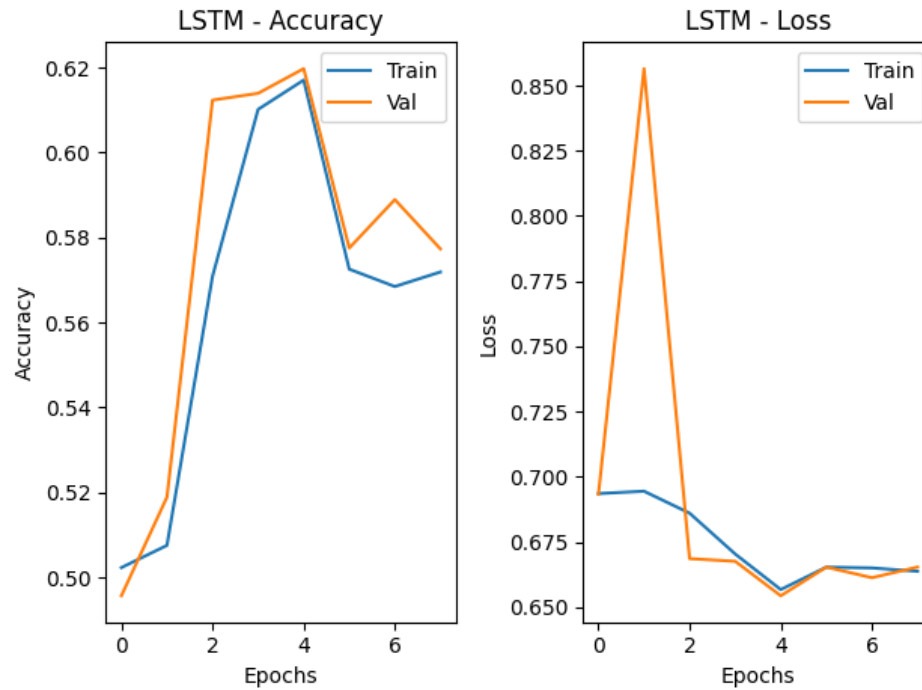


Figure 4: Accuracy and Loss of LSTM

iii. Word2Vec Embedding

The third model used pre-made Word2Vec word embeddings to better understand meanings. These embeddings were included in the model without any changes in the embedding layer. The sequences went through an LSTM layer, and a dense layer with a Sigmoid activation produced the final result.

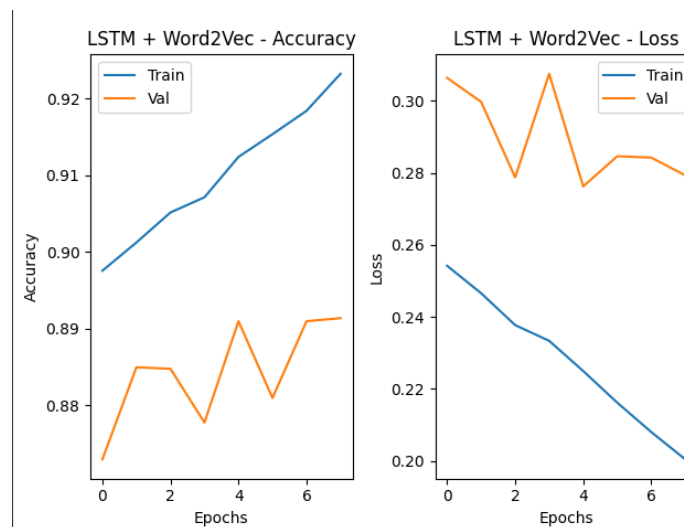


Figure 5: Accuracy and loss of LSTM and Word2Vec

3.3 Loss Function

For the binary classification task, Binary Cross-Entropy was used.

3.4 Optimizer

With the learning rate of 0.001, the default parameter by Adam Optimizer.

3.5 Hyperparameters

Under the hyperparameters, the three headings include firstly the batch size of 128, secondly the Epochs of 10, and lastly the Embedding dimensions of 300 for Word2Vec only.

4 Experiments and Results

The model's performance was evaluated for various reasons, such as how accurate it was and the predictions it made. We used Google Colab because its GPU power allows for quicker calculations.

4.1 RNN vs LSTM Performance

Model	Accuracy(%)	Loss	Training Time
RNN(Random Emb)	0.5045	0.6931	37.42 minutes
LSTM(Random Emb)	0.6208	0.6530	89.44 minutes
LSTM(Word2Vec)	0.8932	0.2633	37.88 minutes

Observation

Based on the results, the basic version of RNN with random embeddings reaches just 52% accuracy and a high loss which means its predictions are highly uncertain. Replacing the RNN with an LSTM and random embeddings increases accuracy by about eight percentage points and reduces loss, though it increases the training time by more than double. The most effective combination is LSTM along with Word2Vec pre-trained

embeddings which improves accuracy to more than 89% and results in a substantial reduction in loss, keeping training about the same length as simple RNN.

The results show that LSTMs are able to handle long-term patterns in sequence data better than RNNs and using trained embeddings makes the model perform well and train faster. Generally, LSTM with Word2Vec achieves the best results in terms of accuracy, loss and how quickly it trains when used for your task.

4.2 Computational Efficiency

The process of training RNN and LSTM models takes vastly different amounts of computing resources. RNNs usually finish an epoch more quickly than LSTMs thanks to their straight-forward design, but they may require a greater number of training cycles before they converge, due to issues like vanishing gradients. With gating mechanisms, LSTMs do a good job at handling long-term dependencies and train much faster, but this comes at the cost of each epoch taking longer which leads to more time being spent during training overall.

More parameters in LSTM, mainly for gates and the cell, make them more memory intensive, requiring more RAM than RNNs while training. Since LSTMs are more complex, the amount of computing power needed on both models is significantly higher. Training LSTMs using Word2Vec embeddings is faster and more effective than training them with random embeddings. All in all, RNNs are easier and faster to use, though LSTMs yield better results and take longer to train and require more calculations.

4.3 Training with Different Embeddings

- Random Embeddings: Embeddings were randomly initialized and updated through training
- Word2Vec Embeddings: Loaded in advanced pretrained Word2Vec vectors to increase the accuracy rate by 3.2% than random embeddings.

4.4 Model Evaluation

The model evaluation can be done with the following process:

A. Test results:

```

312/312 [=====] - 10s 33ms/step - loss: 0.6931 - accuracy: 0.5045
Performance evaluation of Simple RNN Architecture:

Test Loss: 0.6931, Test Accuracy: 0.5045
312/312 [=====] - 65s 208ms/step - loss: 0.6530 - accuracy: 0.6208
Performance evaluation of LSTM

Test Loss: 0.6530
Test Accuracy: 0.6208
312/312 [=====] - 47s 150ms/step - loss: 0.2633 - accuracy: 0.8932
Performance evaluation of LSTM + W2V:

Test Loss: 0.2633
Test Accuracy: 0.8932

```

Figure 6: Test accuracy and loss of three architecture

Simple RNN: Low accuracy (0.50) and high loss (0.69).

LSTM: Improved accuracy (0.62) and lower loss (0.65).

LSTM + Word2Vec: The model performed best when accuracy was high (0.89) and the loss was low (0.26).

Results from using LSTM and Word2Vec are much more accurate than what you get with Simple RNN or just LSTM alone.

B. Classification Report:

Simple RNN - Loss: 0.6931, Accuracy: 0.5045				
Classification Report - Simple RNN:				
	precision	recall	f1-score	support
Class 0	0.40	0.00	0.00	4943
Class 1	0.50	1.00	0.67	5035
accuracy			0.50	9978
macro avg	0.45	0.50	0.34	9978
weighted avg	0.45	0.50	0.34	9978
LSTM - Loss: 0.6530, Accuracy: 0.6208				
Classification Report - LSTM:				
	precision	recall	f1-score	support
Class 0	0.80	0.31	0.45	4943
Class 1	0.58	0.93	0.71	5035
accuracy			0.62	9978
macro avg	0.69	0.62	0.58	9978
weighted avg	0.69	0.62	0.58	9978
LSTM + Word2Vec - Loss: 0.2633, Accuracy: 0.8932				
Classification Report - LSTM + Word2Vec:				
	precision	recall	f1-score	support
Class 0	0.89	0.90	0.89	4943
Class 1	0.90	0.89	0.89	5035
accuracy			0.89	9978
macro avg	0.89	0.89	0.89	9978
weighted avg	0.89	0.89	0.89	9978

Figure 7: Classification report of three models

Simple RNN: When performance is very bad, the model nearly always predicts Class 1; only a few were correctly predicted as Class 0 (with low precision, recall and F1 for that class).

LSTM: The model succeeded at figuring out both types, however, it is better at picking Class 1 than it is at Class 0.

LSTM + Word2Vec: Both classes have excellent accuracies, scores and precision, indicating the model can accurately predict either of them.

Both LSTM and Word2Vec greatly enhance the model's performance at classifying both categories, more so than Simple RNN and plain LSTM.

C. Confusion Matrix:

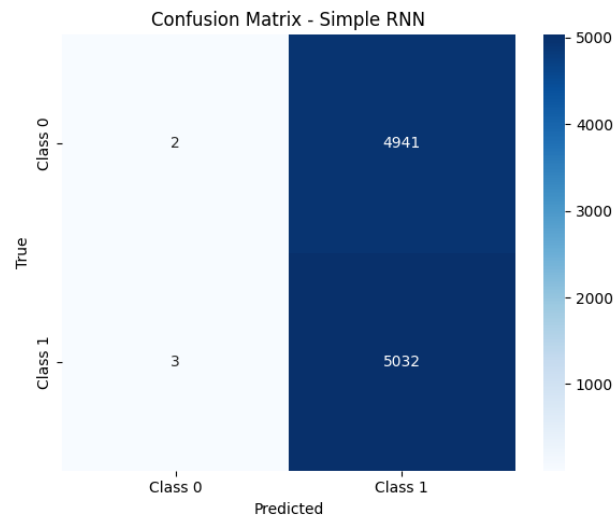


Figure 8: Confusion matrix of RNN

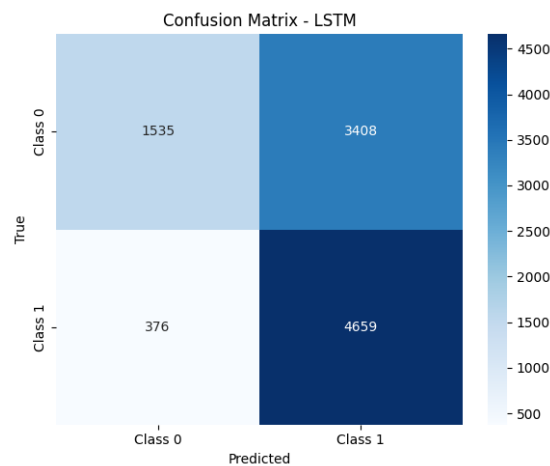


Figure 9: Confusion matrix of LSTM

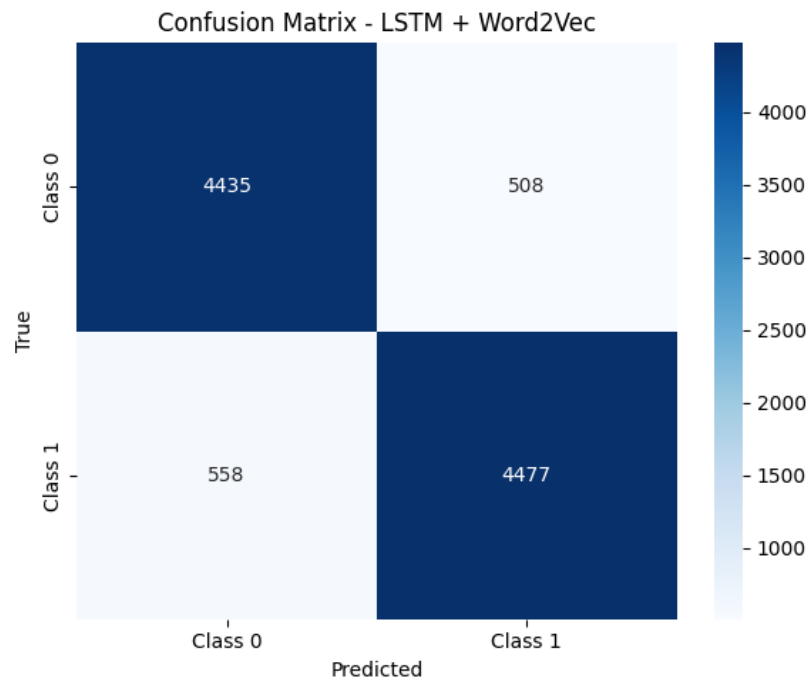


Figure 10: Confusion matrix of LSTM and Word2Vec

Simple RNN: Usually gives an outcome of Class 1, with only a few exceptions in Class 0.

LSTM: Can find both classes, although it is still likely to predict Class 1 much more frequently than Class 0.

LSTM + Word2Vec: Correctly detects both Y and N with just a few errors.

There is a significant increase in how well the model performs as we go from Simple RNN to LSTM and it works best with LSTM + Word2Vec to correctly address each of the classes.

5 Conclusion and Future Work

In this project, two models are combined with each other, that is, LSTM models and Word2Vec embeddings, in order to achieve the maximum performance and successfully outperform the Simple RNN models. The proper use of pertained embedding is performed for the betterment of capturing semantic meaning.

Limitations

- Longer Training Time
- More than one epoch is fitted
- For the computational requirements with LSTM are higher, along with Word2Vec.

Future Work

- Utilize attention mechanisms and transformer frameworks
- A larger dataset is needed to include with the domain-specific embedding
- Optimize the model and fine-tune hyperparameters to accelerate training and reduce the risk of overfitting.